

Wie doctus technisch funktioniert. ■

Eine gründliche Tour durch Komponenten, Datenflüsse und Implementierungsprinzipien — vom Source Connector über COBOL-Analyse und RAG bis zum Multi-Panel-Frontend.

Zielgruppe: Entwicklerinnen und Entwickler, die doctus erweitern, betreiben oder in Kundenumgebungen integrieren.

doctus übersetzt gewachsene Informationen in navigierbares Wissen.

Das System hält drei Arten von Informationen zusammen: originale Quellen, daraus extrahierte Struktur und semantischen Kontext. Jede Nutzerfunktion kombiniert mindestens zwei davon.

ORIGINAL	STRUKTUR	BEDEUTUNG
Quellen COBOL-Dateien, Copybooks, Git-Historie sowie begleitende Dokumente aus Confluence, Jira, WebDAV, Ordnern und Uploads.	Entitäten & Kanten Programme, Sections, Paragraphs, Datenfelder und SQL-Blöcke sowie Aufrufe, Kopien und Verwendungen.	Chunks & Vektoren Durchsuchbare Textabschnitte mit Metadaten und BGE-M3-Embeddings für semantisches Retrieval.

Arbeitsprinzip: Antworten und Visualisierungen sollen nicht vom Originalbestand entkoppelt sein. Dateien, Seiten und physische Zeilen bilden deshalb wiederverwendbare Anker.

■ SYSTEM ARCHITECTURE

Sieben Compose-Services bilden die lokale Laufzeit.

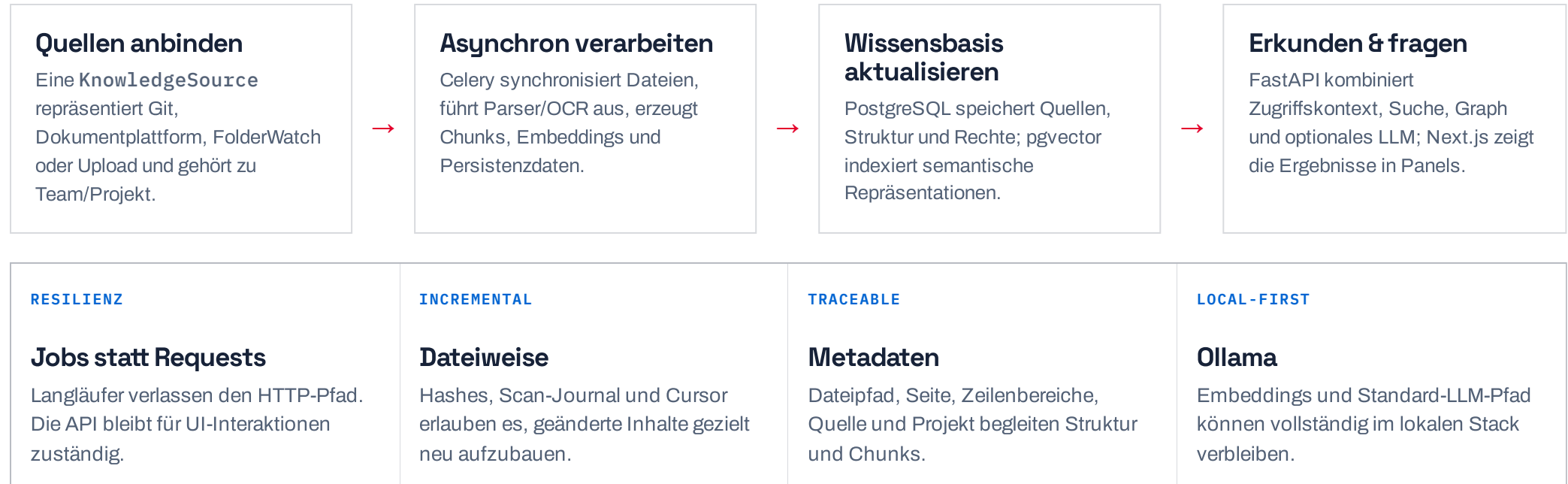
Frontend, API, Hintergrundverarbeitung, Datenhaltung, Queue und KI sind getrennt deploybar, laufen aber in einem gemeinsamen Compose-Netz.

01 · UI	02 · API	03 · Worker	04 · Scheduler	05 · State & KI
frontend Next.js-Server und React-Arbeitsfläche. Next.js 16React 18	backend-api HTTP API, Sessions, Retrieval, Chat und Autorisierung. FastAPISQLAlchemy	parser-worker Synchronisiert, parst, chunkt und embedded Quellen. CeleryPython	parser-beat Startet wiederkehrende Aufgaben wie FolderWatch. Celery Beatperiodisch	db · Valkey · Ollama Persistenz, Broker/Cache und lokale Modellinferenz. PostgreSQLpgvector BGE-M3

Compose-Services: frontend, backend-api, parser-worker, parser-beat, db, redis (Valkey) und ollama.

■ END-TO-END FLOW

Vom Eingang einer Quelle bis zur Arbeit im Browser.



SOURCE CONNECTORS

Wissensquellen werden über ein gemeinsames Ingestion-Modell angebunden.

QUELLE	VERARBEITUNGSPRINZIP	ERGEBNIS IM SYSTEM
Git	Bare Mirror und branch-isolierte Worktrees; Delta-Synchronisierung und dateiweise Verarbeitung.	Repositorydateien, COBOL-Entitäten/Kanten, Code-Chunks und Embeddings.
Confluence / Jira	Connector lädt fachliche Seiten bzw. Vorgänge und normalisiert sie als Dokumentinhalt.	Dokument-Chunks mit Quelle, Titel und Metadaten für Suche und Chat.
WebDAV / FolderWatch	Dateien werden heruntergeladen bzw. read-only aus /watched gelesen; Scanintervall steuert Auto-Sync.	Text-/OCR-Ergebnis, Chunks, Embeddings und Fortschrittsstatus.
Lokaler Upload	Datei wird im Uploadpfad persistiert und durch die Dokumentpipeline verarbeitet.	Projektgebundene Knowledge Source und referenzierbare Inhalte.

Gemeinsame Abstraktion: KnowledgeSource trägt Verbindungsdaten, Sync-Status, Fortschritt, Branch, Cursor und fachliche Kontextnotiz. Dadurch verhalten sich Quellen im Workspace einheitlich.

Celery organisiert jede zeitintensive Verarbeitung als nachvollziehbaren Job.

Warum Worker?

Clone, OCR, Parsing und Modellaufrufe können lange dauern. Sie dürfen weder Browser-Requests blockieren noch den API-Prozess in eine instabile Rechenrolle drücken.

01 Broker

Valkey ist Redis-kompatibel und übergibt Aufgaben an den Parser-Worker.

02 Scheduler

Celery Beat startet periodische Aufgaben, etwa FolderWatch-Scans.

03 Jobstatus

Quellen und Jobs speichern Fortschritt, Meldungen und Zeitinformationen für UI und Diagnostik.

Verarbeitungsfolge pro Datei

01 Inhalt einlesen

Dateityp bestimmen, Text extrahieren; gescannte PDF können über Tesseract OCR laufen.

02 Struktur / Text zerlegen

COBOL nutzt den spezialisierten Parser; andere Inhalte verwenden generisches Chunking.

03 Embedding erzeugen

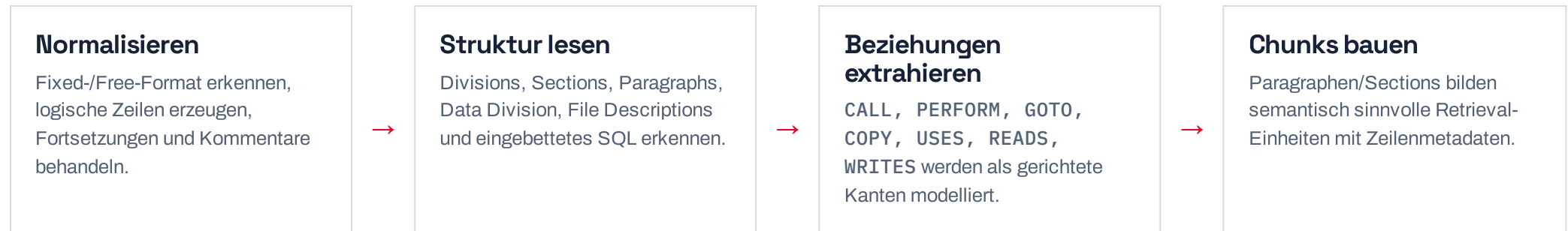
Chunks werden über Ollama mit `bge-m3` vektorisiert; Sub-Batches steuern den Modellaufruf.

04 Transaktional speichern

Chunks, Entitäten und Kanten werden aktualisiert; veränderte Chunks behalten nachvollziehbare Links.

Die Codeanalyse baut eine deterministische Strukturkarte des Bestands.

Der Parser arbeitet lokal und regelbasiert. Er interpretiert den Quelltext, ohne ein LLM zum Parsen einzusetzen, und bewahrt physische Zeilen als stabile Navigationseinheit.



Zentrale Invariante: Copybooks werden nicht in den Programmtext expandiert. Sie bleiben eigene Quellen; die Nutzung wird über COPY-Kanten und vererbte Feldindizes modelliert.

■ PARSER OUTPUT

Entitäten und Kanten bilden den technischen Code Graph.

ENTITY TYPES	HIERARCHY	IDENTITY	SOURCE ANCHOR
CodeEntity program, copybook, section, paragraph, data_item, file_fd und sql_block.	parent_id Programm → Section → Paragraph sowie Programm/Copybook → Datenfelder werden als Hierarchie im selben Entity-Modell gespeichert.	qualified_name Ein qualifizierter Name wie PROGRAM.SECTION.PARAGRAPH stabilisiert Deep-Links und Upserts.	Datei & Zeile Jede Entität trägt Datei sowie Start-/Endzeile; Metadaten halten z. B. PIC, Level, OCCURS oder SQL-Typ.

KANTENART	BEISPIEL	AUFLÖSUNGSBEREICH
CALL / COPY	Programm ruft anderes Programm auf; Programm bindet Copybook ein.	Global innerhalb einer Quelle; Ziel kann erst im Nachlauf vorhanden sein.
PERFORM / GOTO	Kontrollfluss zwischen Paragraphen/Sections.	Lokal im Programm, damit gleichnamige Ziele nicht falsch verknüpft werden.
USES / DEFINES	Paragraph oder SQL-Block verwendet Datenfeld.	Lokal bzw. mit Copybook-Feldvererbung.
READS / WRITES	Datei-/Datensatzoperationen.	Als semantische Beziehung für Analyse und Graphen.

■ PERSISTENCE & RESOLUTION

Der Parser ist DB-frei; Persistenz und Kantenauflösung sind eigene Prozessschritte.

Pass 0 • Index

Der Git-Prozess erstellt einen Copybook-Index über den Bestand, damit **COPY** und geerbte Felder adressierbar werden.



Pass 1 • Persist

`persist_parse_result()` schreibt Entity- und Kantenresultate pro Datei per Upsert.



Pass 2 • Resolve

Globale Kanten bekommen ihren konkreten Zielknoten, sobald der Zielbestand im Index vorliegt.



Graph verfügbar

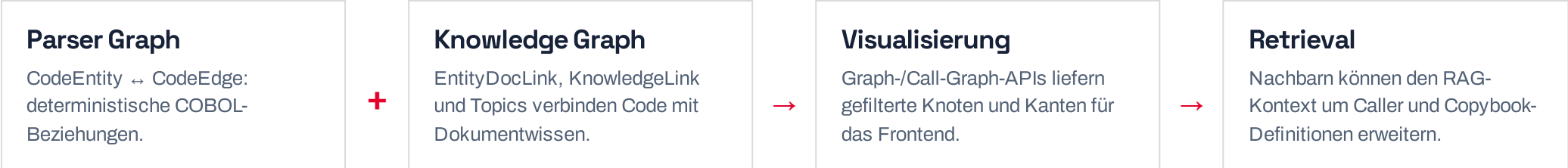
Call Graph, Referenzen, Suche und graph-erweitertes Retrieval können auf Kanten zugreifen.

Warum Upsert statt Löschen? Ein Reparse einer Datei darf keine eingehenden Kanten aus unveränderten Dateien kaskadierend entfernen. Nur verschwundene Entities werden gezielt gelöscht; Kanten der gerade verarbeiteten Quelle werden ersetzt.

Der Knowledge Graph ist kein separater Dienst, sondern eine Projektion der relationalen Wissensbasis.

Graphansichten und Call Graphs werden aus `CodeEntity`, `CodeEdge`, Dokumentlinks, Topics und Projekt-/Quellengrenzen aufgebaut. PostgreSQL bleibt dabei System of Record.

NODES	EDGES	TRAVERSAL
Technische und fachliche Objekte Codeentitäten, Dokumente, Quellen, Projekte und Topics können als Knoten in unterschiedlichen Ansichten auftreten.	Explizite Beziehungen Parserkanten beschreiben Codeabhängigkeiten. Entity-Document- und Knowledge-Links ergänzen fachliche Zusammenhänge.	Kontextabhängige Sicht Call Graphs nutzen 1–3 Hops und Filter. Sichtbarkeit wird gegen Projekt- und Teamzugriff geprüft.



Dokumente werden zu metadatenreichen Chunks — nicht zu einem unstrukturierten Textblock.

Extraktion

01 Text first

PDFs mit Textlayer und Office-/Textformate werden nativ gelesen.

02 OCR fallback

Bildbasierte PDFs können Seiten rasterisieren und Tesseract OCR für Deutsch/Englisch verwenden.

03 Inhalt schützen

Chunk-Inhalte werden in der Datenbank als verschlüsselte Werte gespeichert.

Chunk-Metadaten

01 Position

`file_path`, Start-/Endzeile und bei Dokumenten Seitenzahl erlauben die Rückkehr zur Fundstelle.

02 Herkunft

`source_id`, `project_id`, Titel und Quelltyp halten Kontext und Berechtigungsbezug fest.

03 Semantik

Ein 1024-dimensionalen Vektor repräsentiert den Inhalt für Ähnlichkeitssuche über `pgvector`.

■ DATABASE OVERVIEW

PostgreSQL + pgvector ist der gemeinsame Speicher für Betriebs- und Wissensdaten.

BEREICH	ZENTRALE TABELLEN	AUFGABE
Identität & Zugriff	users, teams, team_memberships, projects, project_memberships	Lokale Identitäten, Organisationseinheiten und projektfeine Rechte.
Quellen & Synchronisierung	knowledge_sources, source_scan_files	Connector-Konfiguration, Status, Cursor, Datei-Journal und Fehler-/Fallbackinformation.
Struktur & Retrieval	code_entities, code_edges, document_chunks	Codehierarchie, Beziehungen, Textsegmente und Vektorindex.
Arbeitswissen	entity_doc_links, knowledge_links, topics, topic_nodes	Kuratiert bzw. automatisch erzeugte fachliche Querverbindungen.
Interaktion & Betrieb	chat_sessions, chat_messages, Job-/Diagnosemodelle	Unterhaltungen, Workspace-Snapshots, Feedback und Nachvollziehbarkeit.

Indexierung: `document_chunks.embedding` verwendet einen HNSW-Cosine-Index (`m=16`, `ef_construction=64`). Daneben existieren B-Tree-Indizes für häufige Source-, Project-, File- und Entity-Zugriffe.

Nutzerverwaltung beginnt lokal und wird durch Teams und Projekte verfeinert.

Der Einstieg ist eine lokale Nutzerverwaltung: beim ersten Start wird ein Bootstrap-Superuser erzeugt. Danach verwaltet die Anwendung Nutzer, Teams, Teammitgliedschaften und Projektmitgliedschaften.

USER	TEAM	PROJECT	MEMBERSHIP
Lokale Identität Username, optionale E-Mail/Name, Argon2id-Hash, Aktivstatus, Passwortwechsel-Flag und Loginstatus.	Organisationsgrenze Ein Team gruppiert Nutzer und ist die grundlegende Sichtbarkeitsgrenze für Quellen und Projekte.	Arbeitskontext Projekt ist oberster fachlicher Container. Quellen, Chunks, Entitäten und Kanten werden ihm zugeordnet.	Feingranularer Zugriff Projektmitgliedschaft bestimmt, wer innerhalb eines Teams ein konkretes Projekt verwenden darf.

Admin: Ein globaler Administrator verwaltet Teams/Nutzer. Die Erkennung erfolgt über das konfigurierte Admin-Konzept; Team- und Projektmitgliedschaften bleiben davon getrennte Domänenobjekte.

AUTHENTICATION

Login erzeugt eine signierte, HTTP-only Anwendungssession.

01 · CREDENTIALS Login Request POST /auth/login prüft Benutzername und Passwort gegen die lokale users-Tabelle.	02 · PASSWORTPRÜFUNG Argon2id Der Hash wird verifiziert; bei geänderten Parametern kann ein erfolgreicher Login transparent rehashen.	03 · THROTTLE Redis + DB Fehlversuche fließen in Redis- und DB-Zähler; Sperrzeit folgt der höheren wirksamen Sperre.	04 · SESSION Cookie doctus_session ist signiert, HTTP-only, SameSite=Lax und 14 Tage gültig; Secure bei HTTPS.
---	---	--	--

BAUSTEIN	VERANTWORTUNG
api/auth.py	Login, Logout, Passwortwechsel, aktueller Nutzer und Login-Flow.
core/passwords.py	Hashing, Verifikation und Rehash-Entscheidung.
core/login_throttle.py	IP-/Username-bezogene Zähler und Sperrlogik.
core/auth_dependency.py	Cookie signieren, laden, maximale Sessiondauer prüfen und get_current_user bereitstellen.

■ AUTHORIZATION

Autorisierung kombiniert globale Adminrechte mit Team- und Projektzugriff.

EBENE	FRAGE	MECHANISMUS
Authentifiziert?	Darf die Anfrage grundsätzlich an geschützte Router?	FastAPI-Router verlangen <code>get_current_user</code> ; auth/system bleiben bewusst separat.
Administrator?	Darf der Nutzer organisationsweit verwalten oder Grenzen überblicken?	<code>require_admin</code> schützt z. B. Teams, Nutzer, Topics und Diagnosefunktionen.
Team sichtbar?	Gehört Projekt/Quelle zu einem Team, das der Nutzer sehen darf?	Gemeinsame Teamhelfer filtern Listen und prüfen Einzelressourcen; unsichtbare Ressourcen verhalten sich wie nicht gefunden.
Projektmitglied?	Darf ein Teammitglied dieses konkrete Projekt nutzen?	ProjectMembership ergänzt Team-Sichtbarkeit; Projektadmins verwalten Mitglieder und Access Requests.
Spezialrolle?	Darf ein Nutzer fachlich sensible Aktionen ausführen?	Die Rolle <code>pruefingenieur</code> ist für Compliance-Freigabe vorgesehen; <code>admin</code> verwaltet Mitgliedschaften.

Kontextsichtbarkeit: Codeanalyse kann pro Projekt explizit für den allgemeinen Kontext freigegeben werden. Standardmäßig bleibt Codeanalyse außerhalb des eigenen Projektkontexts verborgen.

■ DATA PROTECTION

Gespeicherte Inhalte und Connector-Geheimnisse bleiben verschlüsselt im lokalen Datenbestand.

FERNET AT REST

EncryptedString

Tokens, Dokument-Chunks, Chat-Nachrichten und relevante Compliance-Inhalte verwenden einen Fernet-basierten Verschlüsselungstyp.

KEY SEPARATION

Zwei Secrets

MASTER_ENCRYPTION_KEY verschlüsselt Daten.
SESSION_SECRET_KEY signiert Anwendungssessions.

NETWORK EXPOSURE

Local services

PostgreSQL, Valkey und Ollama werden im Compose-Standard nur auf dem Host-Loopback veröffentlicht; Container sprechen intern über Service-Namen.

Entwicklungsfolge: Verschlüsselte Textspalten können nicht einfach per SQL-ILIKE durchsucht werden. Pfade, Metadaten und Vektoren bleiben deshalb wichtige Retrieval-/Filtermechanismen; begrenzte exakte Textprüfungen erfolgen nach dem Laden in Python.

RAG recherchiert, prüft und baut das belegte Kontextpaket.

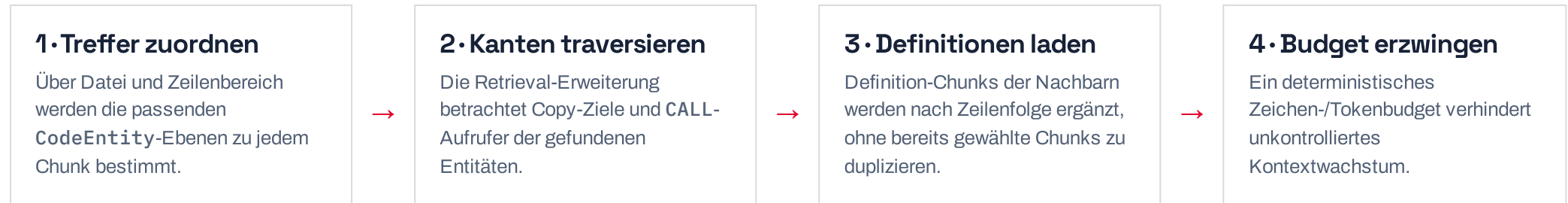
RAG ist die Recherche- und Kontextschicht, nicht die Antwortgenerierung: Sie legt fest, welche zugänglichen Quellen zur Frage passen, und übergibt dem LLM daraus ein begrenztes, referenzierbares Arbeitsmaterial.



VECTOR Semantisch pgvector sortiert Chunks nach Cosine-Distanz zum Query-Embedding.	EXACT Referenzbewusst Seitenzahlen und Abschnittsnummern sind semantisch schwach, aber fachlich präzise; sie werden zusätzlich behandelt.	SCOPED Zugriff zuerst Die Basisabfrage ist bereits auf Projekt/Quelle und Berechtigung eingegrenzt, bevor Retrieval-Ergebnisse entstehen.
---	---	---

Der Graph ergänzt Vektortreffer um technisch benachbarten Kontext.

Vektorähnlichkeit findet den passenden Ausschnitt. Für Codefragen reicht dieser Ausschnitt oft nicht: Ein Treffer in einem Programm benötigt zum Verständnis möglicherweise das verwendete Copybook oder die Aufruferkette.



Resultat: Die Antwort kann den semantisch ähnlichen Text mit dem strukturell notwendigen technischen Umfeld kombinieren. Eine Sichtbarkeitsprüfung filtert Graphnachbarn erneut im Kontext der Anfrage.

LLM: begründen und erklären. RAG: Belege liefern.

Es liest RAG-Treffer nicht bloß vor: Das LLM verbindet die ausgewählten Ausschnitte, erklärt Zusammenhänge, zieht kenntlich gemachte Schlussfolgerungen und strukturiert die Antwort zur konkreten Frage. RAG bleibt dabei der kontrollierte Researchweg.

AUFGABE	LLM-ROLLE	GRENZE ZU RAG / SYSTEM
Einordnen & erklären	Verdichtet mehrere Belege, erläutert Ursache-Wirkung und passt Tiefe, Sprache und Aufbau an die Frage an.	RAG wählt und begrenzt die zugänglichen Belege; das LLM erweitert sie nicht eigenmächtig zur Wissensbasis.
Schlussfolgern	Leitet aus mehreren Ausschnitten eine nachvollziehbare Antwort ab und kennzeichnet Unsicherheit oder fehlende Information.	Keine Primärquelle erfinden oder Vermutung als belegte Tatsache ausgeben.
Agentischer Codekontext	Bei Repo-Kontext kann der Agent gezielt Dateiansicht, Codesuche und Entitätssuche nutzen, wenn der erste Kontext nicht genügt.	Werkzeuge liefern weiteres Material; Parser und Graph bestimmen weiterhin die technische Struktur.
Quellen nennen	Verknüpft die Antwort mit passenden Dateien, Zeilen oder Quellentiteln.	Das Backend akzeptiert nur Zitate, die aus Retrieval oder tatsächlich genutzten Werkzeugen stammen.

RAG LIEFERT Belegtes Material Ausschnitte, Herkunft und Zitieranker — nach Zugriff und Relevanz gefiltert.	LLM LEISTET Verstehen & vermitteln Es verdichtet, erklärt und macht Schlussfolgerungen samt Unsicherheit transparent.	SYSTEM SICHERT Nachvollziehbarkeit Parser, Graph und Quellenprüfung halten Struktur und Belege prüfbar.
--	---	---

■ CITATIONS & SOURCE VALIDATION

Eine generierte Antwort wird erst nach Quellenauflösung zur nachvollziehbaren Arbeitsantwort.

Candidate sources

Vektortreffer und tatsächlich genutzte Agent-Werkzeuge liefern eine Kandidatenmenge.



Answer cites

Die Antwort nennt Datei-/Zeilenreferenzen oder bekannte Quellentitel nach vereinbarten Formaten.



Backend validates

Nur zitierte Kandidaten, die wirklich im Retrieval/Tooling vorkamen, werden als Quellen akzeptiert.



UI links back

Referenzen können wieder zu Datei, Zeile, Dokumentseite oder Kontextansicht führen.

Warum diese Zusatzschicht? Ein Modell kann plausible, aber nicht existierende Pfade nennen. Die Quellenauflösung koppelt sichtbare Belege daher an den tatsächlichen Researchweg statt allein an generierten Text.

Das Frontend ist ein konfigurierbarer Arbeitsraum, kein einzelnes Chatfenster.

Next.js orchestriert eine React-Anwendung mit mehreren gleichzeitig offenen Ansichten. Der zentrale Seitenzustand verteilt Panelkonfiguration, Auswahl, Fokus, Historie und Workspace-Snapshots an spezialisierte Komponenten.

SHELL	CODE	GRAPH	SETTINGS
app/page.tsx Orchestriert Projekt, Panels, Selektionen, Chat- und Workspace-Zustände.	Monaco Editor Zeigt Quelldateien; Klicks und Marker können aus einer Zeile einen referenzierbaren Fokus machen.	Graph views Call Graph und Knowledge Graph visualisieren gefilterte Knoten/Kanten aus API-Ergebnissen.	Admin & Sources Quellen-, KI-, Team-, Nutzer- und Projektverwaltung folgen dem Berechtigungsmodell der API.

Designabsicht: 1–4 Panels erlauben, Code, Graph, Chat und Kontext nebeneinander zu halten. Fixierte Views und Panelhistorien verhindern, dass eine Navigation den Arbeitsstand ungewollt verliert.

Fokusobjekte verbinden UI-Aktionen mit technischen Datenankern.



UI-BAUSTEIN	TECHNISCHE FUNKTION
Code View	Monaco-Dekorationen und Focus-Navigation verbinden Entity-/Zeileninformationen mit dem sichtbaren Originalcode.
References / Call Graph	API-Abfragen liefern gruppierte Inbound-/Outbound-Kanten; Filter und Hop-Anzahl steuern die Projektion.
Chat View	Persistiert Nachrichten, Metadaten, Referenzen und optional Workspace-Snapshots pro Sitzung.
Job Center	Visualisiert den Status von Hintergrundverarbeitung und schafft eine UI-Brücke zwischen Celery und Nutzer.

 DEVELOPER MAP

Die wichtigsten Einstiegspunkte nach Entwicklungsaufgabe.

WENN DU ÄNDERN WILLST ...	BEGINNE HIER	TYPISCHE NACHBARN
COBOL-Syntax / Entitäten / Kanten	<code>parser/cobol/parse.py</code>	<code>source_format.py</code> , <code>divisions.py</code> , <code>data_division.py</code> , <code>procedure.py</code> , <code>xref.py</code> , Golden-Tests.
Persistenz und Nachauflösung	<code>parser/cobol_persist.py</code>	<code>parser/tasks/edge_resolver.py</code> , ORM-Modelle, Migrationen.
Git-/Dokument-Ingestion	<code>parser/connectors/git.py</code> bzw. <code>parser/connectors/</code>	<code>chunk_reindex.py</code> , <code>ollama_client.py</code> , Source-Modelle.
RAG / Chat / Quellen	<code>backend/api/chat.py</code>	<code>services/graph_retrieval.py</code> , Agent-/MCP-Code, Projekte/Teams.
Login / Rollen / Projekte	<code>backend/api/auth.py</code> , <code>core/teams.py</code> , <code>core/projects.py</code>	<code>auth_dependency.py</code> , <code>api/teams.py</code> , <code>api/projects.py</code> .
Arbeitsraum / Panel-UX	<code>frontend/app/page.tsx</code>	<code>frontend/components/</code> , <code>app/services/api.ts</code> , Views und Settings-Tabs.

 SUMMARY

doctus verbindet deterministische Struktur mit semantischer Assistenz.

CODE VERSTEHEN**Parser + Graph**

COBOL wird in stabil referenzierbare Entitäten und gerichtete Abhängigkeiten übersetzt.

WISSEN FINDEN**RAG + Quellen**

Vektor- und Referenzsuche werden mit strukturellem Kontext und validierten Quellen kombiniert.

SICHER ARBEITEN**Local + scoped**

Lokale Datenhaltung, verschlüsselte Inhalte sowie Team-/Projektzugriff formen den Arbeitskontext.

doctus.

Kein Chatbot mit Code-Anhang, sondern ein Arbeitsraum, in dem Originalquellen, strukturelle Beziehungen und semantischer Kontext zusammenspielen.

Weiterführende Repository-Dokumente: [README.md](#), [docs/IMPLEMENTIERUNGSPLAN.md](#), [docs/ENTSCHEIDUNGEN.md](#), [docs/DEPLOYMENT.md](#) und [docs/produkthandbuch-doctus.html](#).